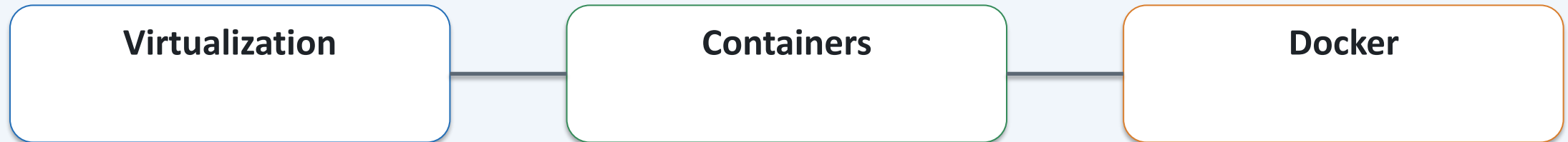


Introduction to Docker & Containerization

From Virtualization and Virtual Machines to Containers



Learning Objectives

By the end of this lecture, students should be able to:

Explain virtualization, virtual machines (VMs), and the role of a hypervisor.

Differentiate virtual machines from containers.

Explain containerization and why containers are lightweight.

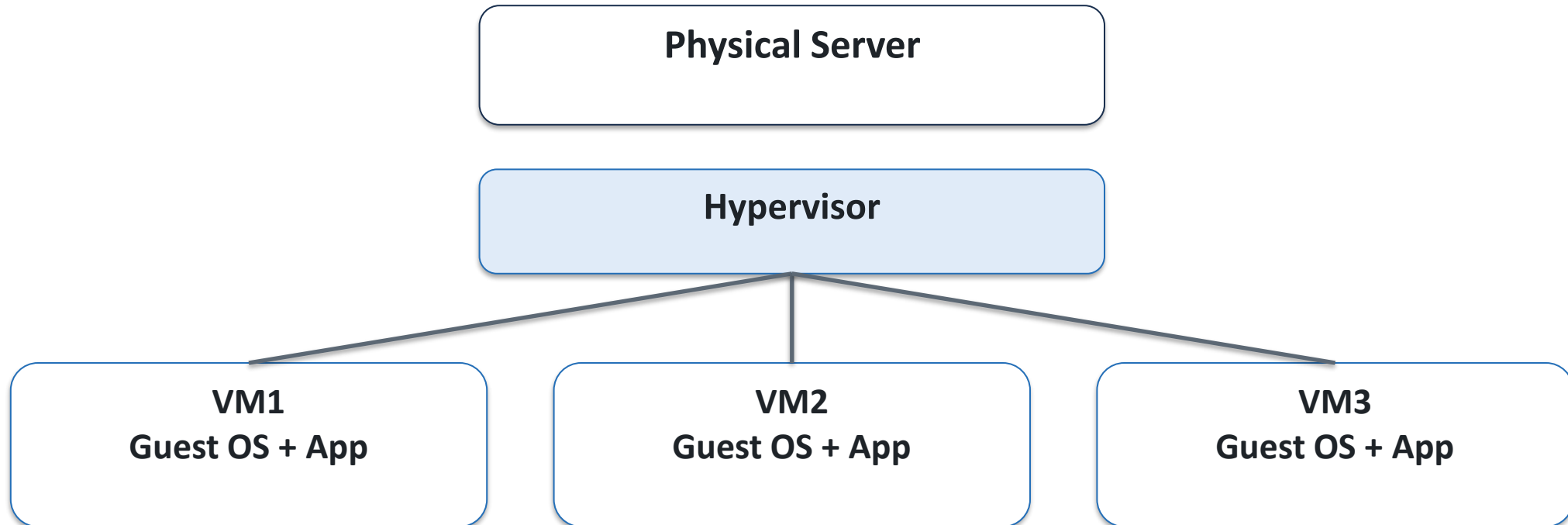
Describe Docker, Docker images, and Docker containers.

Identify key advantages of containers and prepare for basic Docker commands.

What is Virtualization?

Virtualization allows one physical computer/server to run multiple isolated virtual machines.

Each VM behaves like a separate computer with its own operating system and applications.

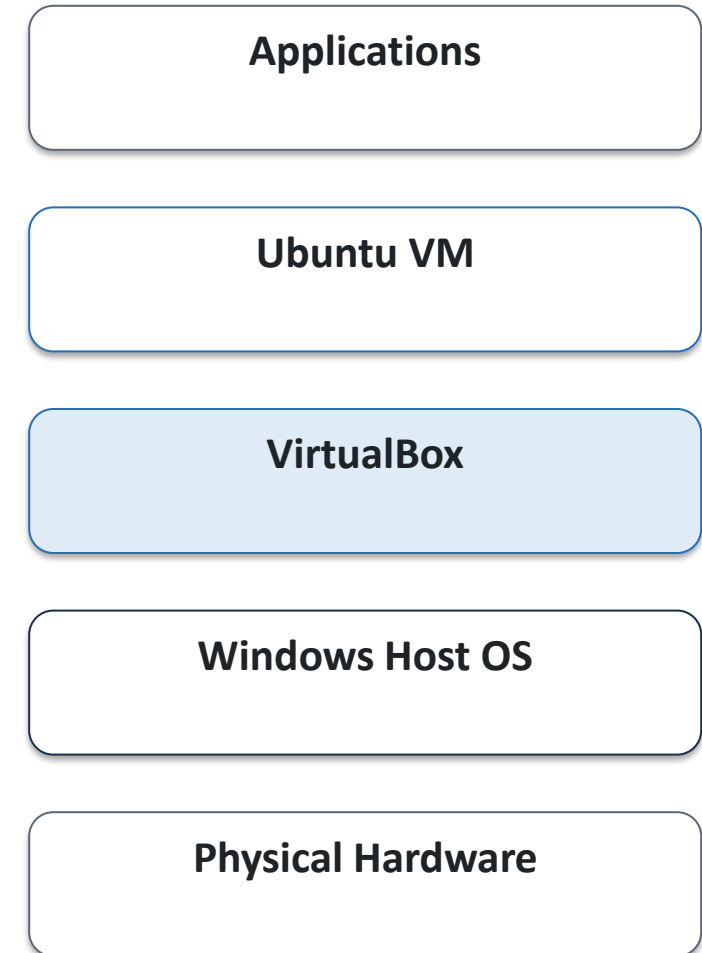


What is a Hypervisor?

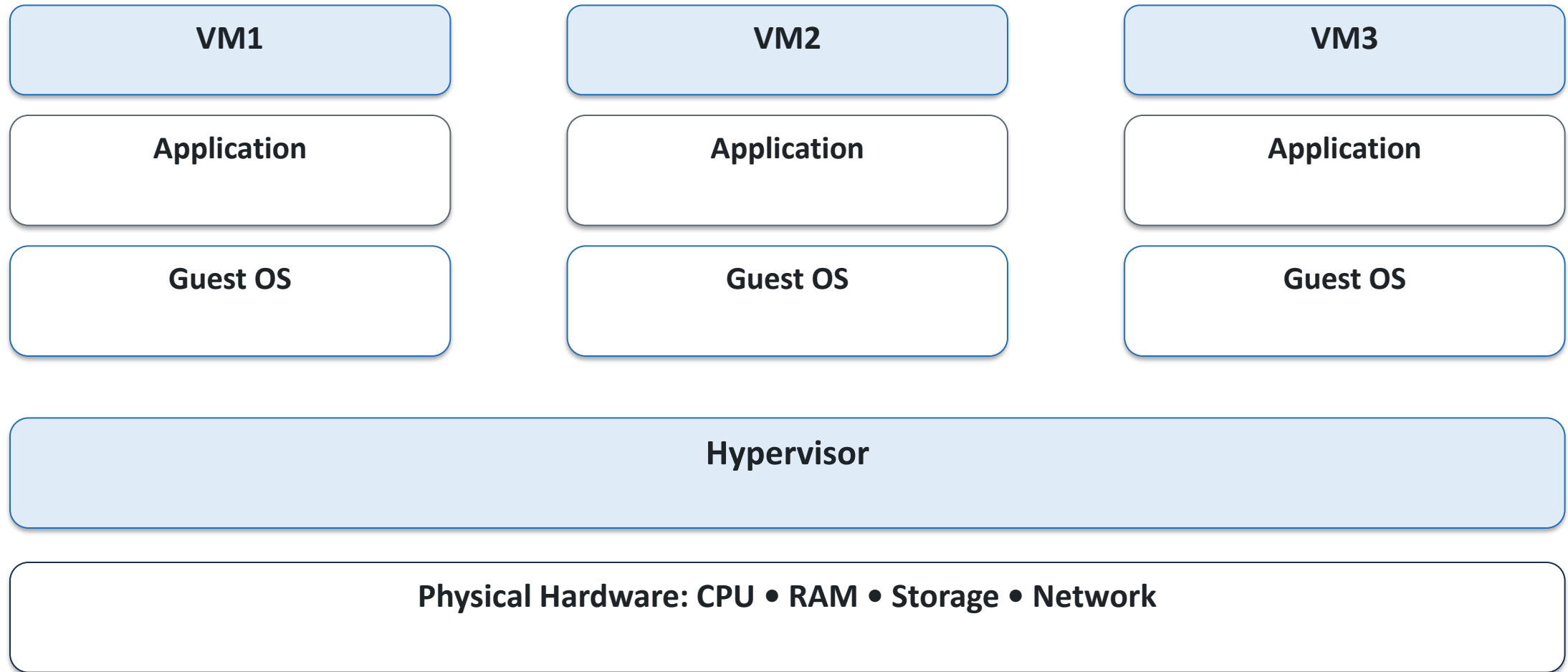
A hypervisor is software that creates and runs virtual machines.

It allocates physical resources such as CPU, memory, storage, and networking to VMs.

Oracle VirtualBox is a hosted (Type 2) hypervisor because it runs on top of a host operating system.



Virtual Machine Architecture



Key idea: each VM includes a full guest operating system.

Why Do We Need Containers?

Applications may require different runtimes, libraries, versions, and configurations.

Running everything directly on one OS can lead to dependency and configuration conflicts.

Containers package an application with what it needs to run, while isolating its processes from other containers.

App A: Python + Lib v1

App B: Python + Lib v2

App C: Other Runtime

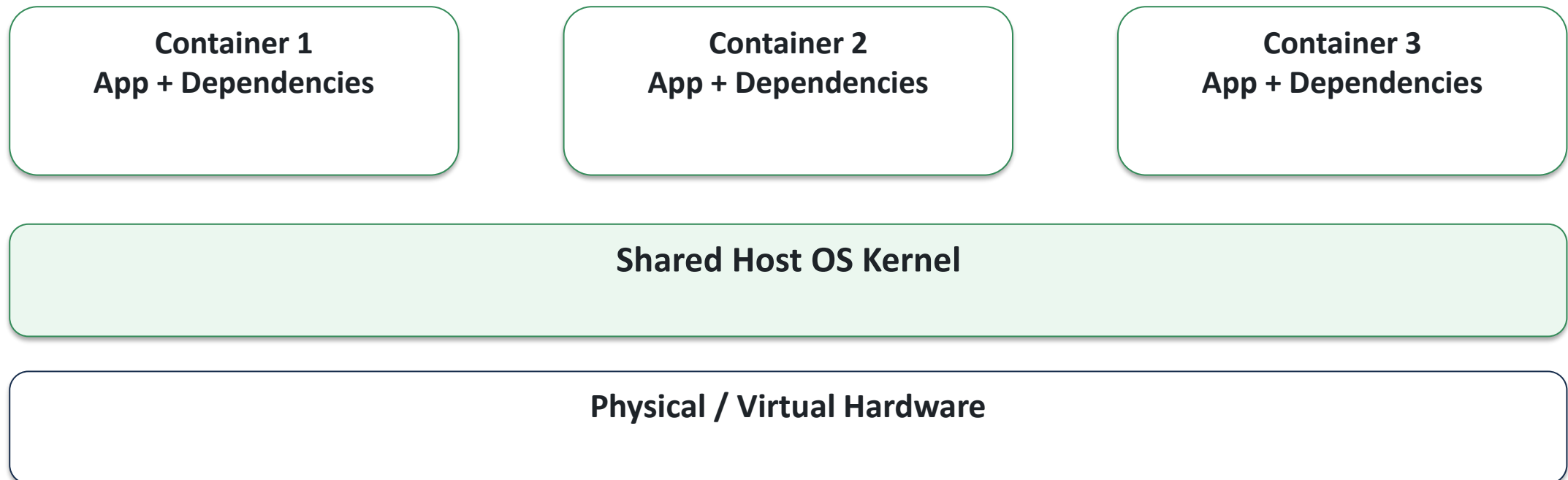
**Containerization
separates environments**

What is a Container?

A container is an isolated environment for running an application and its dependencies.

Linux containers share the host operating system kernel; they do not require a separate guest OS for every container.

A container is not a hypervisor.



Virtual Machines vs Containers

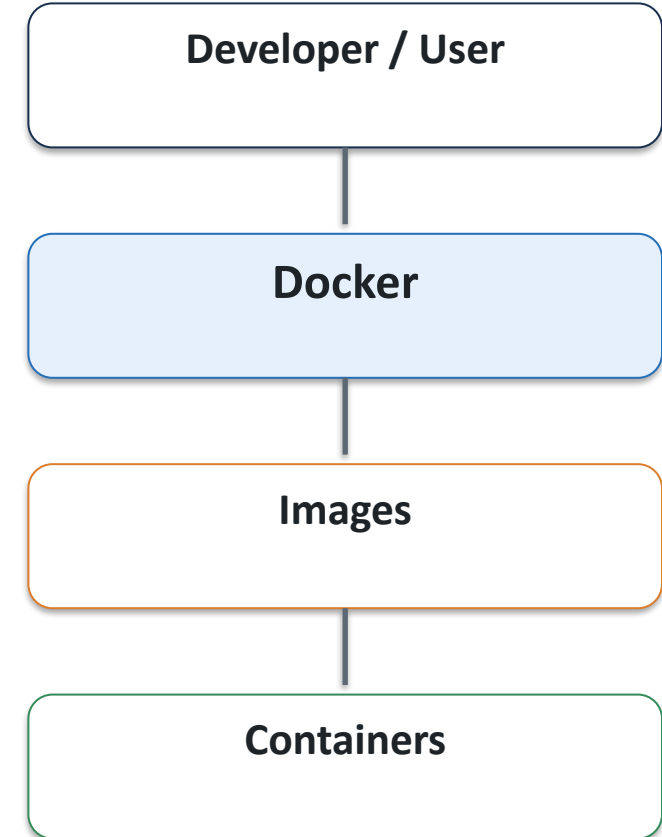
Virtual Machine	Container
Hardware virtualization	OS-level process isolation
Includes a guest OS	Shares the host OS kernel
Typically heavier	Typically lightweight
Usually slower to start	Usually faster to start
Strong machine-level isolation	Application/process isolation
Good for different OS environments	Good for portable application workloads

What is Docker?

Docker is an open platform for developing, shipping, and running applications using containers.

Docker provides tools to build images and create, run, stop, inspect, and remove containers.

Container is the concept/runtime environment; Docker is a popular platform and toolset for working with containers.

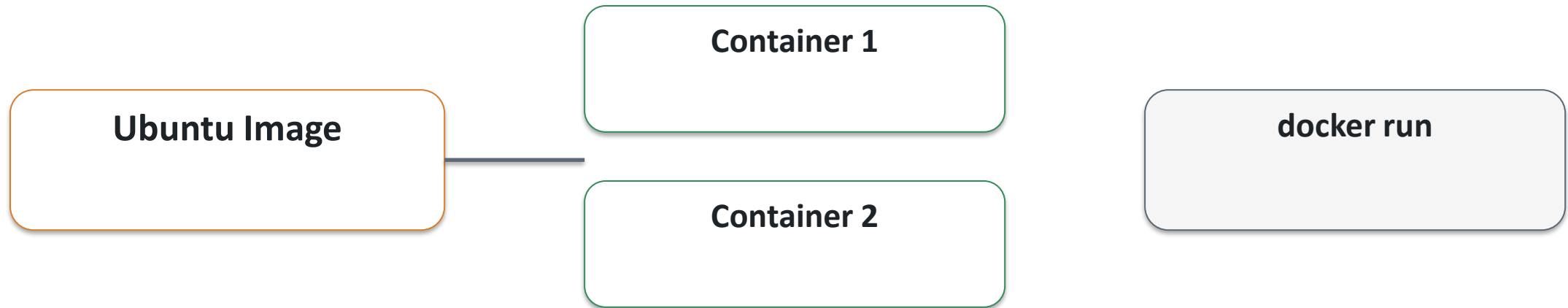


Docker Image and Container

Image = a read-only template/package used to create containers.

Container = a runnable instance of an image.

One image can be used to create many containers.



Why Use Containers?

Lightweight — no full guest OS is required for every container.

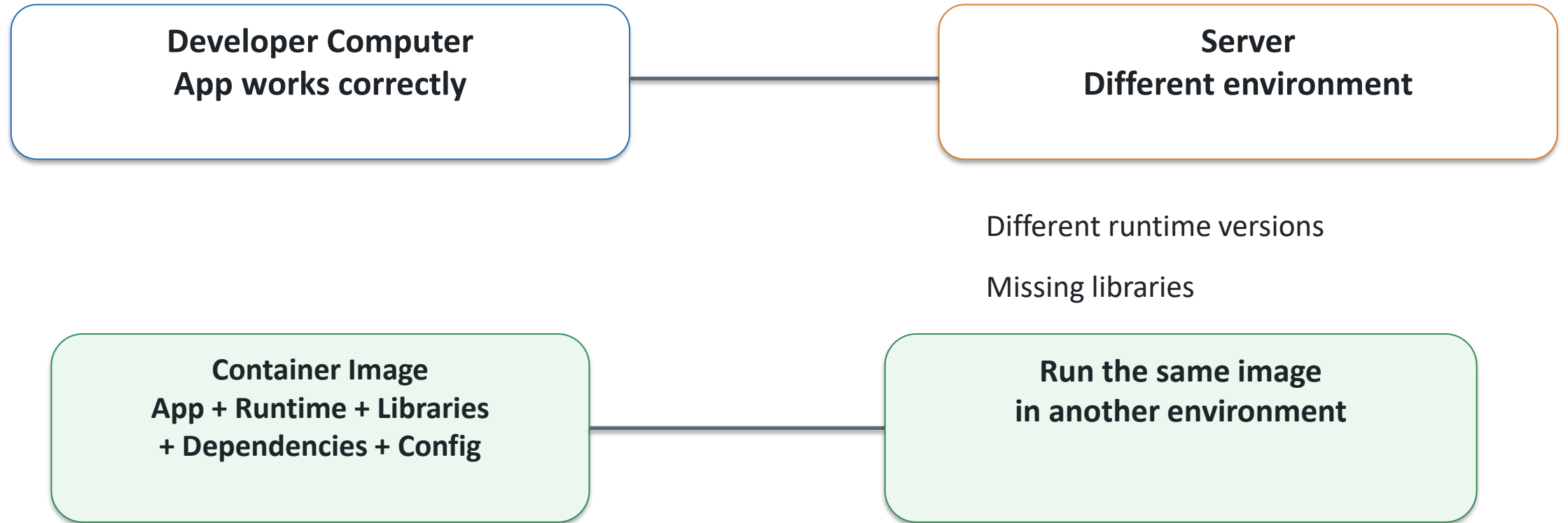
Fast startup — containers start as isolated processes rather than booting a full OS.

Portability — package application code, runtime, libraries, and configuration together.

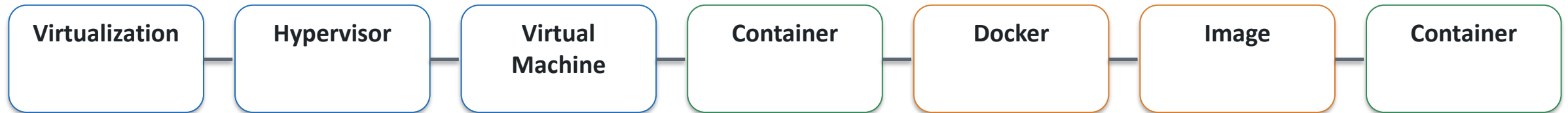
Consistency — the same image can be used across development, testing, and deployment environments.

Resource efficiency — many containers can share the same host kernel.

The "It Works on My Machine" Problem



From Theory to Docker Practical



Next: install/check Docker, pull an image, create containers, and inspect their lifecycle.

```
docker --version
docker pull ubuntu
docker images
docker run ...
docker ps / docker ps -a
```

Key Takeaways

Virtualization uses a hypervisor to create and manage virtual machines.

Each VM normally has its own guest OS.

Containers isolate application processes while sharing the host OS kernel.

Docker is a platform for building and running containers.

Images are templates; containers are runnable instances of images.

Containers are lightweight, portable, fast to start, and resource-efficient.